A Persisted Prediction Pipeline with Fit-Time References and Staged Reliability Reporting

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—ReliableML's `ReliablePipeline` pairs a fitted classifier with a fit-time feature-distribution reference, enabling predictions to be accompanied by unlabeled drift reports. This paper describes the pipeline lifecycle, artifact contents, and the staged availability of reliability analyses. At prediction time, reference comparisons do not require outcomes; evaluation and fairness do, while deterioration requires multiple dated labeled batches and reason auditing requires a labeled holdout. The pipeline preserves an estimator and monitor state but does not automatically clean arbitrary inputs or run every audit after every prediction. Operational safety depends on consistent preprocessing and schema, privacy-aware artifacts, trusted serialization, and follow-up with representative labeled data.**

***Index Terms*—reliable machine learning, prediction pipeline, model monitoring, persistence, deployment**

I. Introduction

Reliable deployment is a lifecycle problem. A model's fit-time reference can support comparisons with new observations, but input similarity and predictive validity are different quantities [1]. This paper documents the ReliableML wrapper's implemented behavior and data requirements [2], [3], emphasizing where the intended operational workflow must pause for labels or other external evidence.

II. Fit-time lifecycle

Given estimator h, training features X_{train}, and labels y_{train}, `ReliablePipeline.fit` fits a `DriftMonitor` on the reference features and then fits h. The monitor retains feature profiles, reference observations, bins/categories, and summary statistics. When reliability monitoring is enabled (the default), the pipeline also saves a reference distribution plot. The monitor compares the input received by the wrapper; callers must ensure the same feature schema, order, units, and representation during training and inference.

The `ReliablePipeline` does not import ATTP or execute general data cleaning. Users remain responsible for cleaning, domain-valid feature construction, resampling boundaries, model selection, and independent evaluation. Keep preprocessing inside the estimator's scikit-learn pipeline whenever it learns from data so that cross-validation fits transformations only on each training fold.

III. Prediction-time outputs

For new features X_{new}, `predict` first returns estimator predictions, then compares the batch with the fitted feature reference. The report includes feature-level drift statistics and an aggregate descriptive similarity indicator, and the pipeline writes a timestamped JSON report containing predictions and a plot. `predict_with_report` returns predictions and the latest reliability report together. This is an unlabeled workflow: it can assess feature-distribution change but cannot establish accuracy, calibration, fairness, or deterioration.

IV. Labeled follow-up analyses

When outcomes are observed, `evaluate(X,y)` computes classification and statistical evaluation and can include group fairness when protected-group features are supplied. `assess_deterioration` takes a history of dated labeled batches and evaluates an accuracy trend. `explain(X)` can explain unlabeled model outputs relative to a background reference. `audit_reasons(X,y)` requires a labeled holdout and reference background and computes predictive-validity and reliance diagnostics. These analyses are separate API calls;

TABLE I
INFORMATION REQUIREMENTS FOR PIPELINE ANALYSES

Analysis	Data requirement
Feature shift	Unlabeled batch and fitted reference
Classification evaluation	Labeled representative batch
Fairness	Labels, protected groups, positive class
Deterioration	Three or more dated labeled batches
Explanation	Background data and target class
Reason audit	Labeled holdout and reference background

they do not all run automatically at every prediction. Their different requirements are summarized below.

V. PERSISTENCE, PRIVACY, AND OPERATIONAL SAFETY

`save` serializes the fitted pipeline with Python pickle; `load` reconstructs it. A profile may retain training observations for reference plots and uncertainty calculations, so the artifact can contain sensitive data and requires access controls, retention policy, and privacy review. Pickle deserialization can execute arbitrary code: only load artifacts from trusted sources. Pin compatible dependencies and record the model, preprocessing, and training-data provenance. Validate schema, units, missing values, and feature order before prediction.

If monitoring is disabled, prediction reports are not produced. A missing report should be treated as disabled or unavailable monitoring, not interpreted as evidence of no shift. Drift thresholds must be empirically reviewed; a low PSI or high similarity indicator is not a correctness probability. Maintain labeled outcome monitoring when feasible and establish human escalation procedures for consequential decisions.

VI. LIMITATIONS

The wrapper does not perform automatic retraining, causal analysis, label acquisition, fairness threshold selection, or all reliability audits at prediction time. A single prediction batch may not support robust statistical inference. Input-distribution comparison is limited by the chosen features, bins, and reference age. Privacy and governance obligations are deployment-specific.

VII. CONCLUSION

The pipeline's central function is to preserve a model and its fit-time feature reference and to report unlabeled distribution comparisons alongside predictions. Outcome-dependent analyses follow when suitable data become available. Keeping these stages explicit prevents users from treating similarity, explanation, or an empty report as proof of reliable predictions.

REFERENCES

[1] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, 2014.

[2] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023.

[3] J. Monchwe, O. Motitswane, and M. Monchwe, *ReliableML*, software, version 0.1.1, 2026. https://github.com/Jabualani/ML-Reliabilty.